

Joshua Vu

CSCI3 740 Computer Security

Lab 2, Tasks 4-6

Task4

```
joshua-vu@joshua-vu-VirtualBox:~/RSA_Lab$ python3 rsa_deterministic.py
--- RSA Keys ---
Public key (n, e): (3233, 17)
Private key (n, d): (3233, 2753)
Enter the filename to read: msg1.txt
Plaintext message:      Transfer 100 dollars to Bob
Message as integer:      [84, 114, 97, 110, 115, 102, 101, 114, 32, 49, 48, 48, 32, 10
0, 111, 108, 108, 97, 114, 115, 32, 116, 111, 32, 66, 111, 98]
Ciphertext 1:            [2159, 2412, 1632, 2235, 1230, 1369, 1313, 2412, 1992, 2906,
624, 624, 1992, 1773, 2185, 745, 745, 1632, 2412, 1230, 1992, 884, 2185, 1992, 524, 218
5, 2570]
Ciphertext 2:            [2159, 2412, 1632, 2235, 1230, 1369, 1313, 2412, 1992, 2906,
624, 624, 1992, 1773, 2185, 745, 745, 1632, 2412, 1230, 1992, 884, 2185, 1992, 524, 218
5, 2570]
Ciphertexts identical:   True
joshua-vu@joshua-vu-VirtualBox:~/RSA_Lab$
```

Why is deterministic encryption dangerous?

Deterministic encryption can be dangerous because encrypting the same message with the same public key will keep giving the same ciphertext. Someone could notice when certain ciphertexts keep showing up and figure out that the same message is being sent, even if they do not know what the message actually says.

Post-Lab Question: Why does encrypting the same message to the same ciphertext violate the intuition behind security?

The point of encryption is to keep information about the original message hidden. If the same message always gives the same ciphertext, it still gives away a pattern that someone could notice. They might not be able to decrypt the message, but knowing when the same message appears again can still reveal useful information.

Task 5

```
joshua-vu@joshua-vu-VirtualBox:~/RSA_Lab$ python3 rsa_guess_attack.py
--- RSA Guessing Attack ---
Public key (n, e): (3233, 17)

Challenge ciphertext: [3165, 1307]

--- Attacker Guessing ---
Candidate: YES
Candidate ciphertext: [99, 28, 2680]
Candidate: NO
Candidate ciphertext: [3165, 1307]
Match found!
Hidden message: NO
```

Why does the guessing attack succeed?

The guessing attack works because plain RSA gives the same ciphertext whenever the same message is encrypted with the same public key. This means someone can encrypt possible answers themselves and compare the results with the ciphertext they are trying to identify. If one matches, they can figure out the message without knowing the private key.

Post-Lab Question: Why is deterministic public-key encryption especially dangerous when the plaintext space is small or predictable? Provide a concrete example.

It is especially risky when there are only a few possible messages because there are not many guesses an attacker has to try. In my experiment, the message could only be yes or no. After encrypting both choices, the ciphertext for no matched the challenge ciphertext. That was enough to figure out the hidden message without actually breaking RSA.

Task 6

```
joshua-vu@joshua-vu-VirtualBox:~/RSA_Lab$ python3 rsa_randomized_demo.py
--- RSA Randomized Encryption Demo ---
Public key (n, e): (3233, 17)

Original message:      YES
Random value 1:        99
Random value 2:        620

Randomized message 1:  YES99
Randomized message 2:  YES620

Ciphertext 1:          [99, 28, 2680, 1175, 1175]
Ciphertext 2:          [99, 28, 2680, 1962, 538, 624]

Ciphertexts identical: False
```

How does randomness improve confidentiality?

Adding randomness makes it so the same original message does not keep producing the same ciphertext. This makes it harder for someone looking at the encrypted data to recognize repeated messages or use the same kind of comparison that worked in the guessing attack.

Post-Lab Question: The original message stayed the same but the ciphertext changed. Explain why this behavior is desirable in secure public-key encryption.

This is useful because seeing two different ciphertexts does not immediately tell someone whether the original messages were the same. In this experiment, I encrypted yes twice, but the random values changed each time and gave me two different ciphertexts. This helps hide patterns and makes it harder to learn information just by comparing encrypted messages.